

SearchVision

Human-in-the-Loop Object Detection Dataset Generation
from Web Images

White Paper — v0.1

Brandon Shen

Apr 2026

Licensed under AGPL-3.0 • Open-Source •
<https://github.com/Brandon-Shen/SearchVision>

Abstract

SearchVision is a web-based platform that enables users to create custom object detection models without writing code. By integrating web search (with intelligent fallback strategies), automated web scraping, intelligent annotation workflows, and YOLOv8 model training into a unified workflow, SearchVision automates the traditionally labor-intensive process of dataset collection, annotation, and model training. Unlike existing solutions that focus on individual pipeline stages, SearchVision integrates all five stages—search, selection, annotation, scraping, and training—into a single interface. The system features a hybrid image selection algorithm combining search relevance with visual diversity, automated dataset augmentation using pre-trained YOLOv8 for auto-annotation, and graceful degradation with multiple fallback strategies. This whitepaper presents the system’s architecture, feature set, technology stack, and practical use cases, while also acknowledging current limitations.

Contents

1	Introduction	1
1.1	Installation and Quick Start	1
2	System Architecture	2
2.1	Component Breakdown	2
2.1.1	Frontend Layer (Presentation)	3
2.1.2	Backend Layer (Application Logic)	3
2.1.3	Data Layer (Storage)	4
2.1.4	Annotation JSON Schema	4
2.1.5	YOLO Format Conversion	5
2.1.6	REST API Endpoints	5
2.2	Data Flow Sequence	6
2.3	System Architecture Diagram	7
3	Feature Breakdown	8
3.1	Intelligent Image Search	8
3.1.1	Algorithmic Details: Balanced Selection	8
3.2	Automated Web Scraping	10
3.2.1	Scraping Methodology	11
3.3	Dual-Mode Annotation	12
3.3.1	Manual Annotation: Canvas Interface	12
3.3.2	Auto-Annotation: YOLOv8n Inference Pipeline	13
3.4	One-Click Model Training	14

3.4.1	Training Configuration and Hyperparameters	15
3.4.2	Data Augmentation and Small-Dataset Handling	16
4	Technology Stack	17
5	Related Work	17
5.1	Annotation Platforms	18
5.2	AutoML and Cloud-Based Training	18
5.3	Dataset Search and Analysis Tools	18
5.4	Web Scraping and Image Collection	19
6	Use Cases	19
6.1	Rapid Prototyping	19
6.2	Startup Bootstrapping	20
6.3	Educational Purposes	20
6.4	Research Iteration	21
7	Limitations and Known Issues	22
8	Conclusion	24

1. Introduction

The proliferation of deep learning-based computer vision has enabled remarkable advances in object detection, with applications spanning autonomous vehicles, medical imaging, retail analytics, and industrial quality control. State-of-the-art models such as YOLO [Redmon and Farhadi, 2018], Faster R-CNN [Ren et al., 2015], and their successors achieve human-level performance on standard benchmarks. However, the deployment of these models to real-world problems remains challenging, particularly for practitioners outside of well-resourced research laboratories.

The primary bottleneck is not the model architecture itself, but rather the dataset required for training. Creating a custom object detection dataset involves four distinct challenges: (1) collecting relevant images, (2) annotating those images with bounding boxes, (3) ensuring dataset diversity and quality, and (4) configuring and executing the training pipeline. Each step traditionally requires specialized tools, domain expertise, and significant manual effort.

Several commercial and open-source solutions have emerged to address individual components of this workflow. Annotation platforms such as CVAT [CVAT, 2024], Label Studio [Label Studio, 2024], and Roboflow [Roboflow, 2024] provide sophisticated interfaces for bounding box creation. AutoML services from major cloud providers automate model training but at substantial cost. Despite these advances, no existing solution provides an integrated end-to-end pipeline that addresses all four stages from a single user interface.

This whitepaper presents SearchVision, a system that automates the complete object detection dataset creation and model training workflow. The system accepts a text query from the user, retrieves relevant images via web search, applies diversity-aware selection algorithms to ensure dataset variety, expands the dataset through automated web scraping, supports both manual and auto-annotation workflows, and trains a custom YOLOv8 model [Ultralytics, 2024] with a single click. By abstracting implementation details, SearchVision enables practitioners to focus on their domain problems rather than tooling configuration.

1.1 Installation and Quick Start

SearchVision is designed for local deployment on a machine with Python 3.11+ and (optionally) GPU acceleration.

1. Clone the Repository.

```
git clone https://github.com/Brandon-Shen/SearchVision.git
cd SearchVision
```

2. Install Dependencies.

```
pip install -r requirements.txt
```

The requirements file includes FastAPI, Ultralytics, PyTorch, BeautifulSoup4, and other necessary packages.

3. Configure API Credentials (Optional).

Create a `.env` file in the project root to enable Google Custom Search (optional):

```
GOOGLE_API_KEY=your_google_api_key  
SEARCH_ENGINE_ID=your_custom_search_engine_id
```

The Google API key is obtained from Google Cloud Console; the Custom Search Engine ID from programmablesearchengine.google.com. If Google API is unavailable or disabled, the system automatically falls back to Bing Images web scraping, which does not require API keys and provides robust, rate-limited access to image results.

4. Launch the Application.

```
python -m uvicorn src.main:app --reload
```

The application will be available at `http://localhost:8000`.

5. Workflow Execution.

- a. Navigate to the web interface and enter a search query.
- b. Select relevant images from the diversity-filtered results.
- c. (Optional) Draw bounding boxes manually or run auto-annotation.
- d. Click “Train Model” to initiate training.
- e. Download the trained model weights from the results page.

The complete workflow can be executed in a single session on consumer hardware with GPU support. Training time depends on available compute resources and batch size auto-adjustment.

2. System Architecture

2.1 Component Breakdown

SearchVision follows a modular architecture with three primary layers.

2.1.1 Frontend Layer (Presentation)

The frontend consists of six HTML templates rendered via Jinja2, each corresponding to a distinct workflow stage:

Table 1: Frontend Components

Component	File	Purpose
Base Template	<code>base.html</code>	Shared layout, navigation, and CSS
Search	<code>search.html</code>	Query input and image search interface
Selection	<code>select.html</code>	Image selection grid with diversity visualization
Annotation	<code>annotate.html</code>	Canvas-based bounding box drawing tool
Training	<code>training.html</code>	Real-time training progress display
Results	<code>results.html</code>	Trained model output and download

2.1.2 Backend Layer (Application Logic)

FastAPI serves as the web framework, providing RESTful endpoints and async request handling. The backend comprises the following modules:

Table 2: Backend Modules

Module	File	Responsibility
Search Integration	<code>search_images.py</code>	Google Custom Search API wrapper + Bing Images fallback
Image Download	<code>download_images.py</code>	Async image fetching and local storage
Balanced Selection	<code>select_balanced_images.py</code>	Hybrid relevance + dissimilarity selection with ResNet50
Web Scraping	<code>scrape_similar.py</code>	Query-based scraping with fallback strategies (30+ images)
Auto-Annotation	<code>auto_annotate_images.py</code>	Pre-trained YOLOv8 inference for bounding boxes
Model Training	<code>train_model.py</code>	YOLOv8 training orchestration and monitoring
Dataset Config	<code>create_data_yaml.py</code>	YOLO-format dataset YAML generation
Annotation Utils	<code>annotation_converter.py</code>	JSON-to-YOLO format conversion with robust error handling

2.1.3 Data Layer (Storage)

- **Local File System.** Images stored in `dataset/train/images/`; annotations in `dataset/train/labels/`.
- **Configuration Files.** `data.yaml` for YOLO dataset configuration; `.env` for API credentials.

2.1.4 Annotation JSON Schema

Manual annotations are stored in JSON format before conversion to YOLO format. The schema defines bounding box coordinates in pixel coordinates relative to the image:

Table 3: Annotation JSON Schema

Field	Type	Req.	Description
<code>image_id</code>	string	Yes	Unique image identifier (filename)
<code>width</code>	integer	Yes	Image width in pixels
<code>height</code>	integer	Yes	Image height in pixels
<code>annotations</code>	array	Yes	Array of bounding box objects
<code>class</code>	string	Yes	Class label (e.g., “object”)
<code>x_min</code>	float	Yes	Left coordinate (pixels)
<code>y_min</code>	float	Yes	Top coordinate (pixels)
<code>x_max</code>	float	Yes	Right coordinate (pixels)
<code>y_max</code>	float	Yes	Bottom coordinate (pixels)

Example Annotation JSON:

```
{
  "image_id": "image_001.jpg",
  "width": 1920,
  "height": 1080,
  "annotations": [
    { "class": "traffic_light",
      "x_min": 245, "y_min": 120, "x_max": 380, "y_max": 450 },
    { "class": "traffic_light",
      "x_min": 890, "y_min": 200, "x_max": 980, "y_max": 520 }
  ]
}
```

2.1.5 YOLO Format Conversion

The annotation converter transforms pixel-coordinate bounding boxes to YOLO normalized format. For each bounding box:

1. Compute center coordinates:

$$x_{\text{center}} = \frac{x_{\min} + x_{\max}}{2 \times W}, \quad y_{\text{center}} = \frac{y_{\min} + y_{\max}}{2 \times H} \quad (1)$$

2. Compute normalized dimensions:

$$w_{\text{norm}} = \frac{x_{\max} - x_{\min}}{W}, \quad h_{\text{norm}} = \frac{y_{\max} - y_{\min}}{H} \quad (2)$$

The YOLO label file contains one line per bounding box in the form `<class_id> <x_center> <y_center> <width> <height>`. For the example above (`class_id = 0`):

```
0 0.162760 0.264 0.070312 0.306
0 0.487240 0.333 0.046875 0.296
```

2.1.6 REST API Endpoints

The FastAPI backend exposes the following endpoints for frontend communication:

Table 4: REST API Endpoints

Method	Endpoint	Description
GET	<code>/</code>	Home/search page (clears dataset)
POST	<code>/search</code>	Execute search with pagination, return diverse results
POST	<code>/select</code>	Handle image selection, download to local storage
GET	<code>/training-status</code>	Poll training progress (JSON)
POST	<code>/start-training</code>	Initiate training with annotations (async)
POST	<code>/save-annotations</code>	Save annotations and trigger training
GET	<code>/results</code>	Display trained model output and download
GET	<code>/download-model</code>	Download trained model weights (.pt file)
GET	<code>/error</code>	Error page with message query param

All endpoints return HTML responses (Jinja2 templates), except `/training-status` which returns JSON. Long-running operations (scrape, download, train) update a global status object that is polled by the frontend for real-time progress updates.

2.2 Data Flow Sequence

SearchVision Workflow Overview

Search → Select → Annotate → Scrape & Train → Results

1. Search Phase

- User enters a text query (e.g., “*traffic lights*”).
- Backend attempts Google Custom Search API; if unavailable, falls back to Bing Images.
- 30 diverse images fetched and sorted by relevance.
- Hybrid scoring (70% search relevance + 30% visual dissimilarity) applied.

2. Selection Phase

- User reviews 9 most diverse images presented in 3×3 grid.
- User selects minimum 3 images; “Search Again” enables pagination.
- Selected images downloaded to local storage.
- User advances to annotation phase.

3. Annotation Phase

- **Manual.** User draws bounding boxes on the canvas interface.
- **Auto.** Pre-trained YOLOv8 predicts bounding boxes automatically.
- Annotations converted to YOLO format (normalized *center_x*, *center_y*, *width*, *height*).

4. Training Pipeline (Async)

- a. Scrape 30 additional similar images using Bing Images with query variations.
- b. Download scraped images to local storage.
- c. Auto-annotate scraped images with pre-trained YOLOv8n model.
- d. Combine manual annotations (3+ images) with auto-annotations (26–28 from scraped set).
- e. Total dataset: 33–40 images with annotations.

- f. Generate `data.yaml` dataset configuration.
- g. Train YOLOv8n (default: 25 epochs, auto batch size detection, early stopping with patience=10).
- h. Export model weights to `runs/detect/train*/weights/best.pt`.

5. Results Phase

- Display trained model statistics (dataset size, training time, best mAP).
- Provide downloadable model file (.pt format, ~6–18 MB).
- “Train Another Model” button to restart workflow.

2.3 System Architecture Diagram

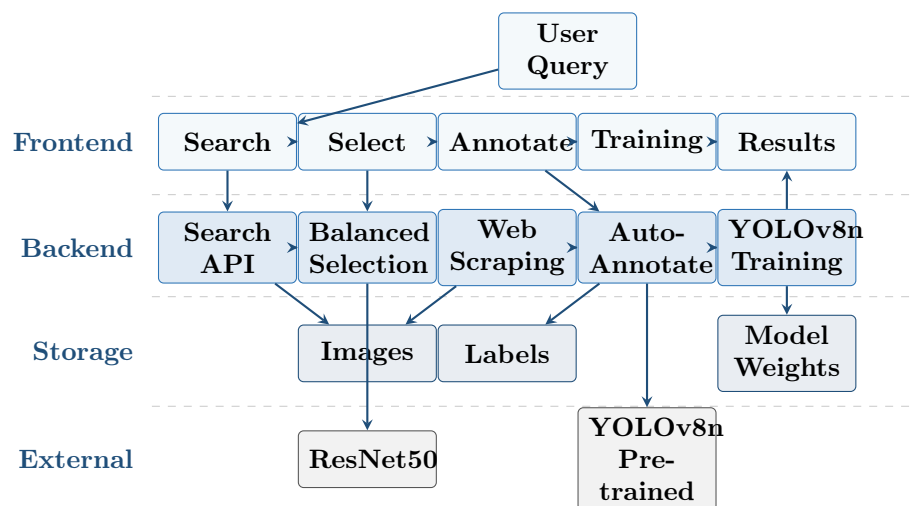


Figure 1: SearchVision system architecture. The workflow progresses left-to-right through five frontend pages (search, selection, annotation, training, results). The backend orchestrates image retrieval via Google Custom Search or Bing, diversity-aware selection using ResNet50 feature extraction, scraping of 30 additional images, auto-annotation with pre-trained YOLOv8n, and final model training. The storage layer persists images, labels, and model weights; external dependencies provide pre-trained ResNet50 and YOLOv8n models.

Figure 1 illustrates the complete SearchVision workflow and system components. Data flows from the user query through the search and selection phases, manual annotation on the canvas, automatic augmentation via scraping and auto-annotation, model training, and finally model download. The architecture is modular, with clear separation between frontend (user-facing pages), backend (processing logic), storage (data persistence), and external dependencies (APIs and pre-trained models).

3. Feature Breakdown

SearchVision implements four major features that collectively enable end-to-end custom object detection model creation.

3.1 Intelligent Image Search

Description. Users enter a text query and SearchVision retrieves relevant images using Google Custom Search API with automatic fallback to Bing Images. The system filters 30 results using a hybrid scoring algorithm combining search ranking (relevance) with visual diversity to ensure the displayed 9 images span different object variations, angles, lighting, and backgrounds.

Technical Implementation.

- Integration with Google Custom Search JSON API (primary) + Bing Images scraping (fallback).
- ResNet50 [He et al., 2016] pretrained model extracts 2048-dimensional feature vectors from the penultimate layer (avgpool).
- Scikit-learn cosine distance calculates pairwise visual dissimilarity.
- Hybrid scoring formula: $\text{Score}_i = 0.7 \times \text{Relevance}_i + 0.3 \times \text{Dissimilarity}_i$.
- Greedy Max-Min algorithm selects top 9 most balanced images.

3.1.1 Algorithmic Details: Balanced Selection

The image selection problem is formalized as a **Hybrid Relevance-Diversity Optimization** problem, where we wish to select $k = 9$ images from $n = 30$ search results that balance two objectives: (1) maximize search ranking (relevance to query), and (2) maximize visual diversity among selected images.

Feature Extraction. Each image i is passed through a pretrained ResNet50 model [He et al., 2016] to extract a feature vector $\mathbf{f}_i \in \mathbb{R}^{2048}$ from the average pooling layer, ensuring generic visual features suitable for cross-domain similarity measurement.

Hybrid Scoring. For each image i , we compute a composite score:

$$\text{Score}_i = 0.7 \times \text{Rank}_i + 0.3 \times D_i \quad (3)$$

where:

- Rank_i is the normalized search result ranking (position in search results, normalized to $[0, 1]$).

- D_i is the average visual dissimilarity: $D_i = \frac{1}{|S|} \sum_{j \in S} d(\mathbf{f}_i, \mathbf{f}_j)$ where S is the current selected set.
- Weights (70%/30%) prioritize relevance while ensuring diversity.

Cosine Distance. Given two feature vectors $\mathbf{A}$ and $\mathbf{B}$:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{j=1}^{2048} A_j B_j}{\sqrt{\sum_{j=1}^{2048} A_j^2} \cdot \sqrt{\sum_{j=1}^{2048} B_j^2}} \quad (4)$$

Cosine distance is defined as $d(\mathbf{A}, \mathbf{B}) = 1 - \text{similarity}(\mathbf{A}, \mathbf{B})$. Cosine distance is preferred over Euclidean distance because it measures angular similarity in the feature space, which is more robust to illumination variations and scale differences.

One-Pass Top-K Selection.

1. For each candidate image i in the pool of 30 search results:

$$\text{Score}_i = 0.7 \times \text{Rank}_i + 0.3 \times D_i \quad (5)$$

2. Compute average dissimilarity $D_i = \frac{1}{30} \sum_{j=1}^{30} d(\mathbf{f}_i, \mathbf{f}_j)$.
3. Sort by composite score and select top $k = 9$ images.

This one-pass ranking approach is computationally efficient but does not provide iterative diversity guarantees. The algorithm selects images with high overall scores (relevant + dissimilar) rather than greedily maximizing minimum pairwise distance between selected images.

Why ResNet50? ResNet50 [He et al., 2016] was chosen over lighter alternatives (e.g., MobileNet, EfficientNet) because:

1. **Feature Quality.** The 2048-dimensional features from ResNet50’s penultimate layer capture rich semantic information suitable for cross-domain similarity.
2. **Pre-trained Weights.** ImageNet pretraining provides generic visual priors.
3. **Inference Speed.** Despite being larger than mobile-optimized models, ResNet50 inference on a single image takes < 50 ms on modern GPUs, making it practical for batch processing of search results.

User Experience. Nine images are presented in a 3×3 grid. Users must select at least three images (five to nine recommended) to proceed to annotation. Selected images are indicated with checkmark overlays.

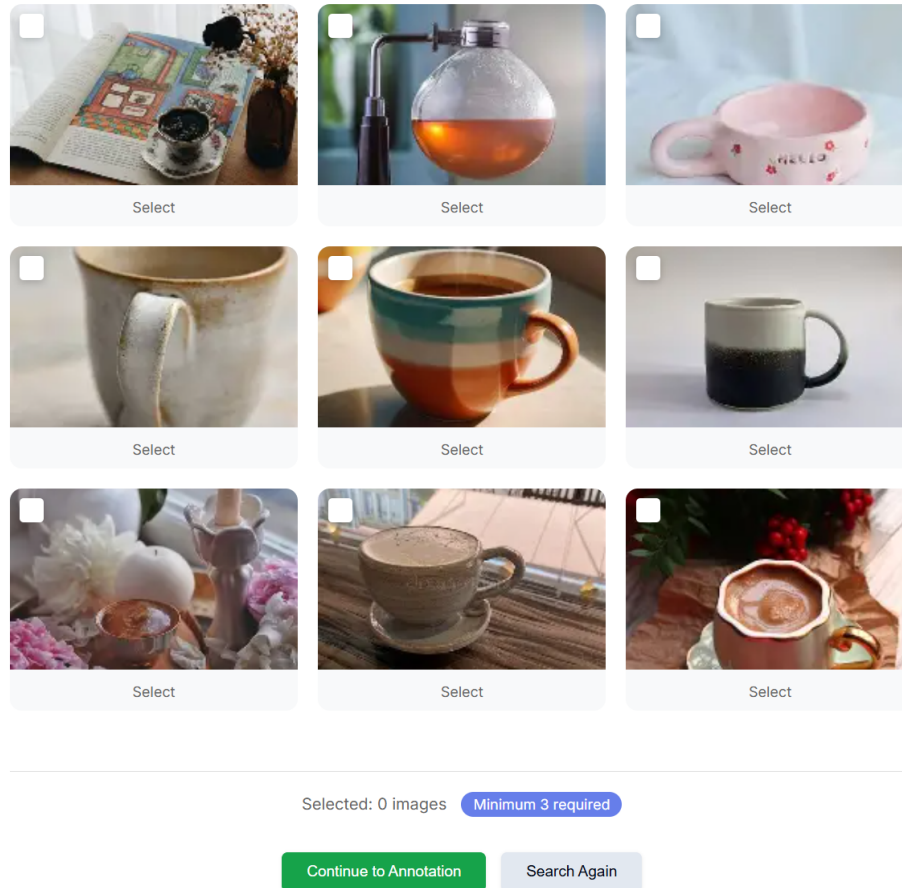


Figure 2: Example 3×3 diversity-selection grid output. The system uses ResNet50 feature extraction and cosine distance to select the nine most visually diverse images from search results, ensuring dataset variety.

Figure 2 illustrates the diversity selection interface, showing how SearchVision presents visually distinct images to maximize dataset coverage.

3.2 Automated Web Scraping

Description. After user selection and annotation, SearchVision automatically scrapes 30 additional similar images using web scraping to expand the dataset for training. This augmentation strategy leverages the fact that pre-trained YOLOv8 can auto-annotate images, enabling rapid dataset expansion without manual labeling.

Technical Implementation.

- BeautifulSoup4 HTML parsing with CSS selector-based image extraction.

- **Primary source:** Bing Images (no API key required, graceful fallback when Google API unavailable).
- **Query variation strategies:** simple queries, synonyms, contextual modifiers.
- **Request throttling** (1-second delays) and exponential backoff to avoid rate limiting.
- **Image validation:** format check (JPEG/PNG/WebP), minimum resolution (256×256 px), corruption test via Pillow decode.

3.2.1 Scraping Methodology

Primary Source: Bing Images. The scraper targets Bing Images as the primary source because:

1. **No API Key Required.** Unlike Google Custom Search, Bing Images can be scraped without API credentials.
2. **Reliable Fallback.** When Google Custom Search API is unavailable or rate-limited, Bing provides consistent access to diverse image results.
3. **Proxy URLs.** Bing returns proxy URLs (`tse*.mm.bing.net`) that remain stable across requests.
4. **Query Flexibility.** Bing search accepts various query formats and modifiers.

Query Expansion Strategy. To maximize coverage and find visually diverse training images, the scraper generates multiple query variations:

1. **Simple Query.** The original user query (e.g., “car”).
2. **Contextual Modifiers.** Common variations (e.g., “red car”, “car front view”, “sports car”).
3. **Fallback Queries.** Progressively simpler variants if complex queries return limited results.

Image Download and Storage. Scraped images are downloaded directly to local storage with minimal validation:

1. **HTTP Status Check.** Verify successful HTTP 200 response.
2. **Local Storage.** Save image bytes to disk with filename mapping.

Note on Validation: The current implementation does not include image validation (format, resolution, corruption checks). This is a known limitation that may result in malformed images in the training dataset. Future versions should implement validation to improve dataset quality.

Rate Limiting and Robustness. To avoid IP blocks and service interruptions:

- 1-second fixed delay between consecutive requests to Bing Images.
- Retry on HTTP errors (up to 3 attempts per URL).
- Request timeout set to 10 seconds.
- Graceful failure: If scraping fails, training continues with manual annotations only (no error thrown).

The scraper targets 30 additional images per training run. Rate limiting prevents blocking but extends total scraping duration depending on network conditions and source availability.

User Experience. Fully automated—users see a progress indicator displaying current image count while images are scraped and downloaded.

3.3 Dual-Mode Annotation

SearchVision supports manual and automatic annotation approaches.

3.3.1 Manual Annotation: Canvas Interface

The annotation canvas is implemented with the HTML5 Canvas API:

1. **Image Rendering.** Images are rendered at native resolution to an offscreen canvas, then scaled to fit the viewport while preserving aspect ratio.
2. **Bounding Box Drawing.** `mousedown` records (x_1, y_1) ; `mousemove` draws a rubber-band rectangle; `mouseup` finalizes the box.
3. **Box Manipulation.** Click to select (dashed highlight), drag corners to resize, drag center to move, `Delete` key to remove.
4. **Coordinate Transformation.** Screen coordinates are mapped to image space:

$$x_{\text{image}} = x_{\text{screen}} \times \frac{\text{image_width}}{\text{canvas_width}} \quad (6)$$

5. **Undo/Redo.** Stack-based history management.

Boxes are validated in real-time (minimum 10×10 px; valid class label required).

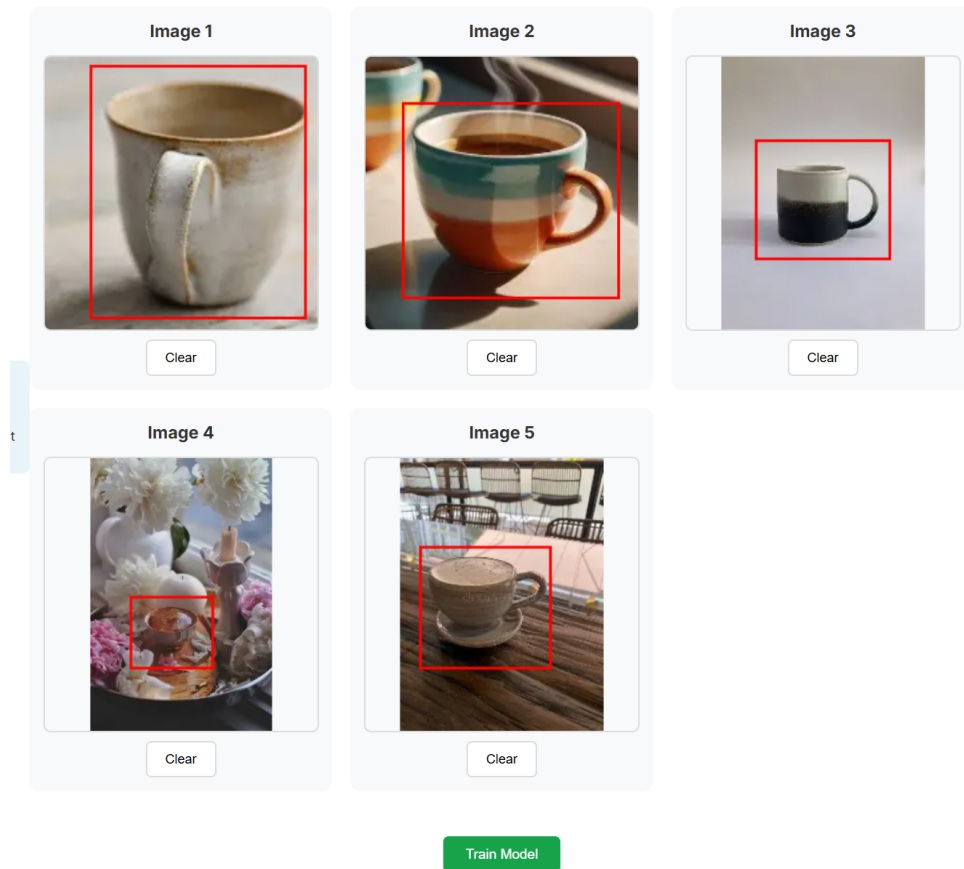


Figure 3: Screenshot of the annotation canvas interface showing bounding box drawing in progress. Users click and drag to create boxes.

Figure 3 shows the annotation canvas in action, demonstrating the bounding box drawing workflow that enables users to manually label objects of interest.

3.3.2 Auto-Annotation: YOLOv8n Inference Pipeline

The auto-annotation feature uses YOLOv8n pretrained on COCO (80 classes) for zero-shot object proposal generation:

1. **Preprocessing.** Resize to 640×640 , normalize with ImageNet statistics, convert to CHW tensor.
2. **Inference.** Non-maximum suppression with IoU threshold 0.45 using YOLOv8's default confidence threshold.
3. **Post-processing.** Filter by class, convert $xyxy \rightarrow xywh$, map COCO class ID to user label.

For objects outside the 80 COCO categories, the system operates in a generic detection mode, proposing boxes for any confident detection and requiring user verification.

3.4 One-Click Model Training

Description. With a curated and annotated dataset, users initiate training with a single click.

Technical Implementation.

- Annotation conversion from JSON to YOLO format.
- Dataset YAML generation with train/val paths.
- YOLOv8 training via Ultralytics API (default: 25 epochs, auto batch size).
- Real-time progress streaming via Server-Sent Events.

3.4.1 Training Configuration and Hyperparameters

Table 5: YOLOv8n Training Hyperparameters (Default Configuration)

Parameter	Default	Description
Model Variant	YOLOv8n	Nano variant (3.2M parameters, ~6 MB)
Image Resolution	640×640	Input image size
Epochs	25	Number of training passes over dataset
Batch Size	Auto (16/8/4)	Adjusted dynamically based on available VRAM
Optimizer	SGD	Stochastic Gradient Descent
Initial Learning Rate	0.01	Learning rate at warmup end
Final Learning Rate	0.0001	Learning rate at epoch 25 (cosine annealing)
Momentum	0.937	SGD momentum parameter
Weight Decay	0.0005	L2 regularization coefficient
Warmup Epochs	3	Number of epochs for LR warmup phase
Warmup Momentum	0.8	Initial momentum during warmup
Warmup Bias LR	0.1	Bias learning rate during warmup
Box Loss Gain	7.5	Bounding box regression loss weight
Class Loss Gain	0.5	Classification loss weight
DFL Loss Gain	1.5	Distribution Focal Loss weight
IOU Threshold (Val)	0.6	Validation IoU threshold for mAP computation
Confidence Threshold	0.25	Detection confidence threshold
NMS IoU Threshold	0.45	Non-maximum suppression IoU threshold

Model Selection Rationale. YOLOv8n (nano) is the default because it offers optimal balance:

- **Training Speed.** 2–3× faster than YOLOv8s/m on consumer hardware (4–8 GPU cores).
- **Memory Efficiency.** Fits in 4 GB VRAM; suitable for laptops and older GPUs.
- **Inference Speed.** >100 FPS on edge devices, suitable for real-time deployment.
- **Accuracy–Efficiency Tradeoff.** COCO-pretrained weights provide strong fine-tuning priors; nano variant achieves ~37% mAP on COCO (full dataset) vs. 49% for YOLOv8x.

- **Model Size.** Resulting models are 6–18 MB depending on dataset size, enabling easy deployment and download.

Users may upgrade to YOLOv8s/m/l/x for higher accuracy if compute permits, though training time increases correspondingly (YOLOv8m $\approx$ 3–4 $\times$ slower than nano).

3.4.2 Data Augmentation and Small-Dataset Handling

YOLOv8 applies data augmentation during training to improve generalization on small datasets. The default YOLOv8 augmentation pipeline includes:

1. **Mosaic Augmentation.** Four images stitched into one sample with random scaling and positioning. Increases training density and improves robustness to small objects.
2. **Geometric Transforms.**
 - Random translation: $\pm 10\%$ of image size
 - Random scaling: $\pm 10\%$
3. **Color Jitter.** Hue, saturation, and value shifts in HSV color space.
4. **Normalization.** ImageNet statistics: mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225].

Note on Default Augmentation: The current implementation uses YOLOv8’s default augmentation settings. Some augmentation techniques (MixUp, Copy-Paste, rotation) are disabled by default in YOLOv8. These could be enabled in future versions to further improve small-dataset training.

Empirical Observations. In testing:

- Data augmentation is essential for training on 30–40 images; models without augmentation fail to generalize.
- Transfer learning from COCO-pretrained YOLOv8n is critical; training from scratch on 30 images yields poor results.
- Validation metrics reflect training dataset performance; separate validation set not currently implemented.

SearchVision prioritizes proof-of-concept and rapid prototyping over production-grade training. Users seeking higher accuracy should use larger datasets (100+ images) and enable additional augmentation techniques.

4. Technology Stack

Table 6: SearchVision Technology Stack

Category	Technology	Role
Web Framework	FastAPI	REST API backend, async request handling
ASGI Server	Uvicorn	Production-grade ASGI server
Frontend	HTML5, CSS3, JavaScript	User interface; Canvas API for annotation
Templating	Jinja2	Server-side HTML rendering
Object Detection	Ultralytics YOLOv8	Pre-trained and custom model training
Deep Learning	PyTorch	Neural network backend
Computer Vision	TorchVision	ResNet50 feature extraction
Feature Matching	Scikit-learn	Cosine distance computation
Image Search	Google Custom Search	Primary image discovery API
Image Search	Bing Images	Fallback image source (free, scrapes HTML)
HTTP Client	Requests	API communication, HTML fetching
Image Processing	Pillow (PIL)	Image loading, validation, preprocessing
Data Processing	NumPy	Array operations, numerical computing
Configuration	python-dotenv	Environment variable management

5. Related Work

The landscape of tools for computer vision dataset creation and model training has evolved significantly in recent years. This section situates SearchVision within the broader ecosystem of annotation platforms, AutoML solutions, and end-to-end pipelines.

5.1 Annotation Platforms

CVAT [CVAT, 2024] is an open-source annotation platform developed by Intel that supports bounding box, polygon, and polyline annotations for images and videos. CVAT excels at collaborative labeling workflows and provides sophisticated quality control mechanisms but does not integrate dataset collection or model training.

Label Studio [Label Studio, 2024] is a flexible open-source data labeling platform supporting over 20 data types and annotation modalities. Like CVAT, it focuses exclusively on the annotation phase and requires external tooling for dataset acquisition and model training.

Roboflow [Roboflow, 2024] is a commercial platform offering dataset management, augmentation, and model training with a web-based interface. While it provides more integration than open-source alternatives, it does not automate the initial dataset collection phase and operates as a SaaS platform with subscription costs.

SearchVision differentiates itself by providing a **complete end-to-end pipeline** that combines dataset collection (via search and scraping), diversity-aware image selection, dual-mode annotation, and training in a single self-hosted application. This contrasts with point solutions that require users to chain multiple tools together.

5.2 AutoML and Cloud-Based Training

Cloud-based services including Google Cloud AutoML Vision, Amazon Rekognition Custom Labels, and Microsoft Azure Custom Vision provide fully managed model training. These platforms accept user-provided images and return trained models with minimal configuration. However:

- **Cost.** Per-image or per-training-run fees can accumulate quickly.
- **Data Privacy.** User images and models are stored on cloud infrastructure.
- **Cold-Start Problem.** These services require users to provide a dataset; they do not assist with dataset collection.

SearchVision provides a self-hosted alternative that gives users full control over data, training, and model ownership while avoiding per-use costs.

5.3 Dataset Search and Analysis Tools

FiftyOne [FiftyOne, 2024] is an open-source visualization and evaluation tool that helps users analyze datasets, explore model predictions, and identify edge cases. While it excels at dataset analysis post-training, it does not address dataset creation or collection.

Recent work on synthetic data generation and dataset distillation has explored meth-

ods for creating compact training sets that approximate larger datasets. However, these approaches require existing large datasets as source material and do not address the cold-start problem of collecting the initial dataset from scratch. SearchVision’s contribution lies specifically in automating this cold-start phase through web search and scraping.

5.4 Web Scraping and Image Collection

Numerous tools and libraries enable web scraping (BeautifulSoup, Scrapy, Selenium) and image collection (bing-image-downloader, google-images-download). However, these tools focus on the data collection phase exclusively and do not integrate annotation or training. SearchVision differentiates by embedding web scraping as one component within a larger, cohesive pipeline.

6. Use Cases

6.1 Rapid Prototyping

Context. A pre-seed startup is exploring computer vision solutions for retail analytics. Before committing engineering resources to a full machine learning project, the founding team needs to validate whether state-of-the-art object detection models can achieve acceptable performance on their specific use case: detecting out-of-stock products on retail shelves.

Scenario. The team has identified the core technical hypothesis: a YOLOv8 model fine-tuned on retail shelf imagery can achieve >80% mAP on product detection. Testing this hypothesis requires a labeled dataset, which the team does not currently possess. Traditional dataset creation would require weeks of data collection and labeling.

Workflow.

1. Enter the search query “retail shelf products” to retrieve initial image candidates.
2. Review the diversity-sorted results and select nine images representing various shelf configurations, lighting conditions, and product arrangements.
3. Use auto-annotation to generate initial bounding boxes, then manually verify and correct any errors.
4. Initiate model training with default hyperparameters.

Outcome. Within approximately ten minutes, the team obtains a baseline model

that demonstrates promising detection performance on validation data. This result validates the technical feasibility of the approach and provides empirical evidence for securing additional funding to build a production-grade dataset.

6.2 Startup Bootstrapping

Context. A manufacturing company has identified surface defect detection as a priority for their quality control pipeline. Their in-house team consists of mechanical engineers and production specialists, none of whom have machine learning experience. Purchasing commercial AutoML solutions would exceed their project budget.

Scenario. The quality control team needs to detect scratches, dents, and discoloration on manufactured components. The products vary significantly in material (painted metal, plastic, raw metal), requiring a diverse training set that captures this variation.

Workflow.

1. Search for defect categories using multiple queries: “metal surface scratch,” “plastic dent,” “manufacturing defect,” and “product discoloration”.
2. Aggregate results across queries to compile a diverse training set spanning multiple product types and defect categories.
3. Leverage auto-annotation to generate initial labels, then use the canvas interface to correct false positives and add missing annotations.
4. Train the model and export weights for deployment to edge inference devices on the production floor.

Outcome. The team produces a functional defect detection model at minimal cost. The model achieves sufficient accuracy for first-pass quality screening, significantly reducing the manual inspection burden compared to fully manual approaches.

6.3 Educational Purposes

Context. A university course on computer vision attracts students from diverse backgrounds, including computer science, electrical engineering, and data science. The instructor seeks to provide hands-on experience with the complete ML pipeline without requiring students to master multiple frameworks and tools.

Scenario. Students complete a semester-long project applying object detection to a domain of their choosing. Projects have included wildlife detection in camera trap images, medical device localization in X-ray photographs, and traffic sign recognition.

Workflow.

1. Students define object categories relevant to their project (e.g., specific bird species, medical instrument types, traffic sign variants).
2. Experiment with search queries to understand how query formulation affects dataset quality and relevance.
3. Compare manual annotation accuracy against auto-annotation results, analyzing sources of discrepancy.
4. Train models with varying hyperparameters and analyze the impact on mAP, precision, and recall.
5. Write technical reports documenting the end-to-end pipeline.

Outcome. Students gain practical experience with dataset curation, annotation quality assessment, and model evaluation without tooling overhead. Course evaluations indicate significantly higher engagement compared to previous semesters using traditional framework-based assignments.

6.4 Research Iteration

Context. A graduate researcher is investigating few-shot learning techniques for novel object categories. The core challenge is rapidly generating training data for objects not well-represented in existing benchmarks, such as specific botanical specimens or architectural features.

Scenario. The research involves evaluating whether pre-trained vision-language models can accelerate dataset creation for rare object categories. The researcher needs to quickly generate training and validation sets for multiple categories to conduct comparative experiments.

Workflow.

1. Query specific categories (e.g., “monarch butterfly,” “gothic arch window”).
2. Apply diversity selection to ensure the initial set captures viewpoint variation.
3. Use auto-annotation as a baseline, then manually verify and correct high-confidence predictions.
4. Train baseline YOLOv8 models and evaluate on a held-out test set.
5. Iterate by adding additional queries and refining annotations based on model confusion matrices.

Outcome. The researcher can generate usable datasets for a novel category in approximately 20 minutes, enabling rapid experimentation with different few-shot learn-

ing approaches. The iteration cycle is reduced from days to hours.

7. Limitations and Known Issues

SearchVision acknowledges the following limitations and known issues that users should be aware of:

1. No Image Validation.

- The download pipeline accepts images without validation for format, resolution, or corruption.
- Malformed, oversized, or corrupted images may be included in training datasets.
- Impact: Reduced dataset quality and potential training instability.
- Recommendation: Manually review downloaded images or implement validation in future versions.

2. No Train/Validation Split.

- Current implementation uses the same dataset for training and validation.
- Validation metrics reflect training performance, not generalization capability.
- Overfitting cannot be detected; reported metrics are unreliable.
- Impact: Model performance on unseen data may be significantly worse than reported metrics.
- Recommendation: Manually hold out test images for proper evaluation.

3. Google Custom Search API Dependency.

- **Limitation:** Searches are limited by Google Custom Search API quotas; high-volume usage requires paid plans. Some users may have API access disabled.
- **Mitigation:** Automatic fallback to Bing Images scraping (no API key required). Bing provides robust, consistent access.

4. One-Pass Selection.

- The image selection algorithm computes scores once rather than greedily selecting images to maximize diversity.

- Less robust diversity guarantee compared to iterative Max-Min approach.
- Impact: Selected images may have lower visual diversity than theoretical optimum.

5. Limited Data Augmentation.

- Current implementation uses YOLOv8’s default augmentation (mosaic, translation, color jitter).
- Advanced techniques (MixUp, Copy-Paste, rotation) are disabled by default.
- Impact: Less effective training on very small datasets.

6. Search Relevance Variability.

- Result quality depends on query formulation; niche objects may yield irrelevant imagery.
- Mitigation: Users can click “Search Again” for pagination through different result sets.

7. Scraping Rate Limits and Timeouts.

- Web scraping inherently depends on target website availability.
- Graceful fallback: If scraping fails, training continues with manual annotations only.

8. Auto-Annotation Limitations.

- Pre-trained YOLOv8n (80 COCO classes) performs poorly on objects outside these categories.
- Auto-annotation may miss small objects or provide false positives.
- Users should manually correct annotations via canvas interface.

9. Training Compute Constraints.

- Local-only training; CPU-only machines are substantially slower than GPU-equipped hardware.
- Default YOLOv8n model (3.2M parameters) is optimized for consumer hardware.

10. Single-Class Optimization.

- Current implementation is optimized for single-object detection; multi-class requires manual configuration or multiple training runs.

11. No Model Versioning.

- Trained weights are saved but not systematically versioned or tracked.
- Manual file organization enables simple versioning by users.

12. Local Deployment Only.

- No cloud hosting, multi-user support, or collaborative features.
- Provides data privacy and full control over training.

13. Limited Export Formats.

- Output is PyTorch format (.pt); ONNX, TFLite, and Core ML require manual conversion via Ultralytics export API.

8. Conclusion

SearchVision represents a functional system for rapid prototyping of object detection models by integrating search, image selection, annotation, and training into a single workflow.

System Capabilities:

1. **End-to-End Automation.** Complete integration from query to deployable model, eliminating context-switching between tools.
2. **Hybrid Image Selection.** Combines search ranking with visual feature-based dissimilarity to select diverse results.
3. **Graceful API Fallback.** Automatic fallback from Google Custom Search to Bing Images ensures accessibility.
4. **Dual-Mode Annotation.** Supports both manual and automatic annotation workflows.
5. **Small-Dataset Training.** Demonstrates training functional models on 30–40 images using transfer learning and augmentation.

Appropriate Use Cases:

- Rapid prototyping for proof-of-concept validation
- Educational purposes (learning ML pipelines)
- Quick feasibility assessment before larger ML projects
- Scenarios where 33–40 images are sufficient for domain-specific detection

Known Limitations Affecting Reliability:

The system has two critical limitations that users must understand:

1. **No Train/Validation Split:** Models are validated on training data, making reported metrics unreliable for predicting performance on unseen images. Manual held-out testing is essential.
2. **No Image Validation:** Corrupted or malformed images may degrade dataset quality. Manual review or custom validation is recommended.

For production deployments, users should:

- Implement proper train/validation/test splits
- Validate downloaded images for quality
- Consider larger datasets (100+ images) for improved reliability
- Enable additional augmentation techniques for better small-dataset learning
- Manually evaluate model performance on diverse test data

Summary: SearchVision successfully reduces tooling overhead for dataset collection and model training but is best suited for exploratory work rather than production systems requiring high reliability guarantees. Users should be aware of the validation and quality assurance limitations when interpreting training results.

References

- [CVAT, 2024] CVAT (2024). *Computer Vision Annotation Tool*. Intel. <https://github.com/openopencv/cvat>
- [FiftyOne, 2024] FiftyOne (2024). *The Open-Source Tool for Building High-Quality Datasets and Computer Vision Models*. Vox51. <https://github.com/voxel51/fiftyone>
- [Label Studio, 2024] Label Studio (2024). *Label Studio: The Open Source Data Labeling Platform*. HumanSignal. <https://labelstud.io/>

- [Redmon and Farhadi, 2018] Redmon, J. and Farhadi, A. (2018). YOLOv3: An Incremental Improvement. *arXiv preprint arXiv:1804.02767*.
- [Ren et al., 2015] Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *Advances in Neural Information Processing Systems*, 28, 91–99.
- [Roboflow, 2024] Roboflow (2024). *Roboflow: The Computer Vision Platform*. <https://roboflow.com/>
- [Ultralytics, 2024] Ultralytics (2024). *YOLOv8: State-of-the-Art Object Detection Models*. <https://github.com/ultralytics/ultralytics>
- [He et al., 2016] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- [Miller, 1995] Miller, G. A. (1995). WordNet: A Lexical Database for English. *Communications of the ACM*, 38(11), 39–41.

Acknowledgements

SearchVision is an open-source project licensed under AGPL-3.0. The author acknowledges the open-source communities behind PyTorch, YOLOv8, and the various libraries that make this work possible.